

华中科技大学

编译原理实验报告

专 业： 计算机科学与技术

班 级： 本硕博 2301

学 号： U202315763

姓 名： 王家乐

电 话： 15391560195

邮 箱： chia.le@foxmail.com

独创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明！

作者签名：王 家 乐

日期：2026 年 6 月 18 日

综合成绩	
教师签名	

目 录

1 工具、语言与平台	1
1.1 实验任务	1
1.2 实验实现	1
2 基于 ANTLR 的词法分析器 (C++实现)	4
2.1 实验任务	4
2.2 词法分析器的实现	4
3 基于 ANTLR 的语法分析器 (C++实现)	6
3.1 实验任务	6
3.2 语法分析器的实现	6
4 静态语义检查 (C++实现)	8
4.1 实验任务	8
4.2 语义检查器的实现	8
5 中间代码生成 (C++实现)	13
5.1 实验任务	13
5.2 LLVM IR 生成器的实现	13
6 基于 LLVM 工具链的目标代码生成 (RISCV64)	15
6.1 实验任务	15
6.2 RISC-V64 目标代码生成器的实现	15
7 总结	17

1 工具、语言与平台

1.1 实验任务

- (1) 编译工具链的使用：掌握 gcc、clang、交叉编译器、qemu 与 Makefile 的基本用法。
- (2) SysY 语言及运行时库：熟悉 SysY 语言和运行时库，并用该语言写一个解决“买卖股票的最佳时机”的程序。
- (3) 目标平台 riscv64 的汇编语言：在 RISC-V 汇编层面实现数组排序函数，理解目标平台调用约定和寄存器使用。

1.2 实验实现

1.2.1 编译工具链的使用

① GCC 编译器的使用

本关需要用 gcc 编译两个 C 语言程序，生成可执行程序。通过本关的必要命令选项：-o 后面的参数指出生成的可执行文件的名称；-D 选项则完成宏定义的功能，以实现 Alibaba 和 Bilibili 之间的“对话”。关键代码如下：

```
gcc def-test.c alibaba.c -DBILIBILI -o def-test
```

② CLANG 编译器的使用

本关需要用 clang 编译器把 bar.c “翻译”成优化的（优化级别 O2）armv7 架构，linux 系统，符合 gnueabihf 嵌入式二进制接口规则，并支持 arm 硬浮点的汇编代码（程序中并没有浮点数）。汇编代码文件名为 bar.clang.arm.s，只需要了解相关命令选项即可：-O2 表示的是优化级别；-target 表明了生成的汇编程序需要遵循的架构和规范等；-S 表示生成汇编程序；-o 与 gcc 一样指定生成文件名称。关键代码如下：

```
clang -O2 -S -target armv7-linux-gnueabihf bar.c -o bar.clang.arm.s
```

③ 交叉编译器 arm-linux-gnueabihf-gcc 和 qemu-arm 虚拟机

本关需要用交叉编译器 arm-linux-gnueabihf-gcc 将源程序“翻译”成 arm 汇

华中科技大学实验报告

编代码，再将汇编代码汇编并与 Sysy2022 运行时库连接，生成 arm 可执行代码，然后用 qemu-arm 虚拟机运行 arm 可执行程序。关键代码如下：

```
# 用 arm-linux-gnueabi-gcc 将 iplusf.c 编译成 arm 汇编代码 iplusf.arm.s
arm-linux-gnueabi-gcc -S iplusf.c -o iplusf.arm.s
# 再次用 arm-linux-gnueabi-gcc 汇编 iplusf.arm.s，同时连接 SysY2022 的运行时库
sylib.a，生成 arm 的可执行代码 iplusf.arm
arm-linux-gnueabi-gcc iplusf.arm.s sylib.a -o iplusf.arm
# 用 qemu-arm 运行 iplusf.arm
qemu-arm -L /usr/arm-linux-gnueabi/ iplusf.arm
```

④ make 的使用

本关需要编写一个 Makefile，使用 make 完成项目的构建。首先是定义各种变量减少输入量：

```
CXX := g++
CXXFLAGS := -std=c++11 -Wall -Wextra -pedantic
INCLUDES := -I./include
SOURCES := main.cc helloworld.cc
OBJECTS := main.o helloworld.o
TARGET = helloworld
```

之后根据生成可执行程序的逆顺序依次写出指令，变量用 \$() 引用，%.cc 和 %.o 能进行后缀匹配，\$^ 和 \$@ 分别表示所有依赖文件和目标文件：

```
all: $(TARGET)
$(TARGET): $(OBJECTS)
    $(CXX) $(CXXFLAGS) $(INCLUDES) $^ -o $@
%.o: %.cc
    $(CXX) $(CXXFLAGS) $(INCLUDES) -c $^ -o $@
```

1.2.2 SysY 语言及其运行时库

需要用 SysY 实现股票最大利润问题。程序只遍历价格数组一次，维护当前最低价格 minPrice 和当前最大收益 maxProfit；每读到一个新价格，先用它与历史最低价计算候选利润，再更新最低价。该算法时间复杂度为 $O(N)$ ，空间复杂度为 $O(1)$ 。

```
int maxProfit(int prices[]){
    int minPrice = prices[0];
    int maxProfit = 0;
    int i=1;
    while(i<N){
```

```
    if (maxProfit < prices[i] - minPrice)
        maxProfit = prices[i] - minPrice;
    if (minPrice > prices[i])
        minPrice = prices[i];
    i = i + 1;
}
return maxProfit;
}
```

1.2.3 riscv 汇编

本关使用 RISC-V64 汇编完成 bubblesort 函数。数组首地址由 a0 传入，长度由 a1 传入；程序使用 t0 保存数组首地址、t1 保存长度、t2/t4 作为外层与内层循环变量，t6 计算 arr[j] 的地址，a2/a3 临时保存相邻元素。比较发现逆序时交换两个元素，全部循环结束后将 a0 置 0 返回。

```
bubblesort:
    mv     t0, a0          # t0 = arr 数组首地址
    mv     t1, a1          # t1 = n 数组长度
    li     t2, 0           # t2 = i = 0, 外层循环变量

.L_outer:
    addi   t3, t1, -1      # t3 = n - 1
    bge    t2, t3, .L_done # if i >= n - 1, 排序结束
    li     t4, 0           # t4 = j = 0, 内层循环变量
    sub    t5, t3, t2      # t5 = n - 1 - i, 内层循环上界

.L_inner:
    bge    t4, t5, .L_next_outer # if j >= n - 1 - i, 结束内层循环
    slli   t6, t4, 2       # t6 = j * 4, 因为 int 占 4 字节
    add    t6, t0, t6      # t6 = &arr[j]
    lw     a2, 0(t6)       # a2 = arr[j]
    lw     a3, 4(t6)       # a3 = arr[j + 1]

    ble    a2, a3, .L_no_swap # if arr[j] <= arr[j+1], 不交换
    sw     a3, 0(t6)       # arr[j] = arr[j + 1]
    sw     a2, 4(t6)       # arr[j + 1] = 原来的 arr[j]

.L_next_outer:
    addi   t2, t2, 1
    j      .L_outer
.L_done:
    mv     a0, 0
```

2 基于 ANTLR 的词法分析器（C++实现）

2.1 实验任务

利用 ANTLR 工具生成 SysY2022 语言的词法分析器，要求输入一个 SysY2022 语言源程序文件，比如 test.cpp，词法分析器能输出该程序的 token 以及 token 的种别。

2.2 词法分析器的实现

词法文件 SysyLex.g4 按类别定义 token。关键字和运算符采用固定字符串规则；标识符用以字母或下划线开头、后接字母数字下划线的规则；整数分为十进制、八进制和十六进制；浮点数覆盖小数点形式和指数形式。

```
// identifier
ID : [A-Za-z_][A-Za-z0-9_]*;

// integer literal
INT_LIT : [1-9][0-9]*
        | '0'[0-7]*
        | '0'[xX][0-9a-fA-F]*
        ;

// float literal
FLOAT_LIT : [+~]? (
    ('.'[0-9]+([eE][+~]?[0-9]+)?[fF]?)
    | ([0-9]+'.'[0-9]*([eE][+~]?[0-9]+)?[fF]?)
    | ([0-9]+[eE][+~]?[0-9]+[fF]?)
);
```

对于词法错误，实验要求像 9ab、2f、096 这类非法串应该报告错误：

```
LEX_ERR : ('0' [1-9a-fA-F]* | [^0-9a-zA-Z]+);
```

主程序 main.cpp 的作用是调用 ANTLR 生成的词法分析器并按实验要求格式化输出结果。程序首先通过 std::ifstream 打开命令行传入的 SysY 源文件，并用该文件流构造 ANTLRInputStream；随后创建 SysyLexer 对输入进行词法分析，再用 CommonTokenStream 收集词法分析得到的 token，并调用 tokens.fill()

华中科技大学实验报告

一次性读取全部 token。之后程序遍历 `tokens.getTokens()` 返回的 token 列表，对每个 token 调用 `getType()` 获取其种别编号、调用 `getText()` 获取源程序中的原始单词串，并在需要报告错误时调用 `getLine()` 获取所在行号。如果 token 类型是 `lexer.EOF`，说明已经到达文件末尾，直接忽略；如果 token 类型是 `LEX_ERR`，则报告词法错误；否则根据 `main.h` 中与词法规则顺序一致的 `tokenTypeName[]` 数组，将 token 编号转换为种别名称，并输出 识别出的单词：种别名称。这样就避免了直接使用 `token->toString()` 带来的内部编号和调试信息输出，使结果符合平台评测要求。

```
int main(int argc, const char* argv[]) {
    std::ifstream stream;
    stream.open(argv[1]);
    ANTLRInputStream input(stream);
    SysyLexer lexer(&input);
    CommonTokenStream tokens(&lexer);
    tokens.fill();

    for (auto token : tokens.getTokens()) {
        auto tokentype = token->getType();
        if(tokentype != lexer.EOF){
            if(tokentype == 43){
                std::cout<<"Lexical error - line "<<token->getLine()<<" : "<<
                    token->getText() << std::endl;
            }
            else std::cout<<token->getText() <<" : " <<
                tokenTypeName[tokentype] <<std::endl;
        }
    }
    return 0;
}
```


3 基于 ANTLR 的语法分析器 (C++实现)

3.1 实验任务

第一关要求利用 ANTLR 生成 SysY2022 语言的语法分析程序, 对于给定的有语法错误的 SysY2022 源程序, 能够由 parser 指出语法错误出现的行号并给出错误信息。第二关在第一关语法规则正确的基础上, 要求对语法正确的 SysY2022 源程序构造并输出抽象语法树 AST。实验框架已经提供了 Sysy.g4、SysyLex.g4、AstVisitor.h、AstVisitor.cpp、main.cpp 以及 AST 打印相关代码, 因此主要工作是补全 SysY2022 的语句文法, 并在 Visitor 模式下补全 while 语句对应的 AST 构造逻辑。

3.2 语法分析器的实现

3.2.1 语法分析器

在 Sysy.g4 中, 前面的编译单元、声明、函数定义、基本类型、数组、初始化列表、表达式优先级等规则已经按照 SysY2022 的语言结构组织好, 本实验需要重点补全 stmt 规则。根据任务说明, Stmt 应覆盖 LVal '=' Exp ';', 可空表达式语句、语句块、if (Cond) Stmt [else Stmt]、while (Cond) Stmt、break;、continue; 和 return [Exp]; 等形式。我的实现中将这此语句分别写成 ANTLR 产生式, 并在每个分支后使用 # assign、# exprStmt、# blockStmt、# ifElse、# while、# break、# continue、# return 标注标签。这样 ANTLR 会为每类语句生成更具体的 Context 类型, 例如 WhileContext、BreakContext、ReturnContext 等, 后续 AstVisitor 中也能直接重写对应的 visit 方法。如果标签名称不一致, ANTLR 生成的方法名会变化, 后面的 AST 构造代码就无法正确覆盖框架中的接口。具体的语句文法实现如下:

```
stmt
: lVal Assign exp Semicolon # assign
| exp? Semicolon # exprStmt
```

```
| block # blockStmt
| If Lparen cond Rparen stmt (Else stmt)? # ifElse
| While Lparen cond Rparen stmt # while
| Break Semicolon # break
| Continue Semicolon # continue
| Return exp? Semicolon # return
;
```

3.2.2 AST 构造

第二关是在第一关语法规则已经补全的基础上，继续完成 AST 构造。框架已经提供了 `AstVisitor` 类和大部分 `visit` 方法，本关实际只需要补全 `AstVisitor.cpp` 中的 `visitWhile` 方法，使语法树中的 `while (Cond) Stmt` 能被转换成 AST 中的 `While` 节点。

实现思路比较直接：`WhileContext` 中可以通过 `ctx->cond()` 取得循环条件，通过 `ctx->stmt()` 取得循环体。分别对这两个子节点调用 `accept(this)`，得到对应的 `Expression *` 和 `Statement *`，再用 `std::unique_ptr` 接管它们的所有权。最后调用 `new While(std::move(cond), std::move(stmt))` 构造循环语句节点，并将其作为 `Statement *` 返回。具体实现如下：

```
antlrcpp::Any AstVisitor::visitWhile(SysyParser::WhileContext *const ctx) {
    auto const cond_ = ctx->cond()->accept(this).as<Expression *>();
    std::unique_ptr<Expression> cond(cond_);
    auto const body_ = ctx->stmt()->accept(this).as<Statement *>();
    std::unique_ptr<Statement> body(body_);
    auto const ret = new While(std::move(cond), std::move(body));
    return static_cast<Statement *>(ret);
}
```

这样，当程序中出现 `while` 循环时，ANTLR 生成的语法树节点会经过该方法转换为 AST 的 `While` 节点，后续 `ast->print(std::cout, 0)` 就可以正确输出循环结构。

4 静态语义检查（C++实现）

4.1 实验任务

本实验要求在已经完成词法分析、语法分析和 AST 构造的基础上，实现 SysY2022 语言的静态语义检查器。语义检查的输入是语法正确的 SysY2022 源程序对应的 AST，输出则是在遍历 AST 时发现的语义错误。与语法分析不同，静态语义检查关注的是程序结构虽然符合文法，但名字、类型、作用域或控制流使用不合法的情况。本关要求能够识别的错误包括：当前作用域变量重复定义、函数形参重复定义、函数重复定义、使用未定义变量、使用未定义函数、函数实参与形参个数或类型不匹配、函数返回值类型不匹配、数组下标不是整数、break 不在循环内、continue 不在循环内，以及对非数组变量使用下标访问。

4.2 语义检查器的实现

首先，在 checker.h 中新增了 current_func_return_type，用于记录当前正在检查的函数返回类型。原有代码检查 return 时通过 find_func() 查找当前函数，不够直接；修改后在进入函数定义时保存返回类型，在访问 return 时直接比较。

```
TYPE current_func_return_type{}; // 当前正在检查的函数返回类型
void Checker::visit(ReturnStmtAST &ast) {
    TYPE ret_type = TYPE::TYPE_VOID;
    if (ast.exp) {
        ast.exp->accept(*this);
        ret_type = current_type.type;
    }
    if (ret_type != current_func_return_type)
        ReportAndExit(ErrorType::FuncReturnTypeNotMatch, "return");
}

void Checker::visit(FuncDefAST &ast) {
    if (!InsertFunc(ast))
        exit(int(ErrorType::FuncDuplicated));
    current_func_return_type = ast.funcType;
}
```

华中科技大学实验报告

```
start_of_new_func = true;
ast.block->accept(*this);
}
```

其次，新增了统一的报错退出函数 `ReportAndExit()`。原有代码中很多地方直接写 `err.error()` 和 `exit()`，修改后用一个函数统一处理，后面数组下标、函数参数、循环语句等错误都调用它。

```
[[noreturn]] void ReportAndExit(ErrorType type, std::string name) {
    err.error(type, name);
    exit(int(type));
}
```

为了完成函数参数匹配检查，还新增了 `SameParamType()`。它不仅比较基本类型，还比较实参和形参是否同为数组，以及数组维数是否一致。对于数组形参第一维为空的情况，用 `-1` 表示，因此比较时跳过这一维。

```
bool SameParamType(const Entry &formal, const Entry &actual) const {
    if (formal.type != actual.type || formal.is_array != actual.is_array)
        return false;
    if (!formal.is_array)
        return true;
    if (formal.array_length != actual.array_length)
        return false;
    // 第一维形参可能是指针维（-1），不要求与实参数值相同。
    // 由于部分框架不做常量求值，维度长度只在双方都可用且形参不是 -1 时比较。
    const auto n = std::min(formal.arlen_value.size(),
                             actual.arlen_value.size());
    for (size_t i = 0; i < n; ++i) {
        if (formal.arlen_value[i] != -1 && actual.arlen_value[i] != 0 &&
            formal.arlen_value[i] != actual.arlen_value[i])
            return false;
    }
    return true;
}
```

在符号表插入部分，`InsertVar()` 相比原有代码补充了数组维度长度的记录，并在非数组变量时显式设置 `array_length = 0`。这样后面检查数组访问和函数数组参数时，可以知道数组还剩多少维。

```
if (!def->arrays.empty()) {
    tmp.is_array = true;
    for (auto &exp : def->arrays) {
```

```
exp->accept(*this);
if (!Expr_int)
    ReportAndExit(ErrorType::ArrayIndexNotInt, *def->id);
tmp.arlen_value.push_back(Expr_value);
}
tmp.array_length = static_cast<int>(def->arrays.size());
} else {
    tmp.is_array = false;
    tmp.array_length = 0;
}
```

InsertFunc() 也做了补充。原有代码只在已有符号是函数时才报函数重复定义，修改后只要当前作用域已有同名符号，就认为函数名冲突并报 FuncDuplicated。同时，数组形参的维度表达式也补充了整型检查，并记录每一维长度。

```
if (table.front().find(*node.id) != table.front().end()) {
    err.error(ErrorType::FuncDuplicated, *node.id);
    return false;
}
```

在 checker.cpp 中，原有代码对 is_inloop 的传递还没有补全，因此无法正确判断 break 和 continue 是否在循环中。修改后在访问语句块、块项、选择语句和循环语句时传递 is_inloop，遇到循环语句时把循环体设置为循环内部。

```
void Checker::visit(BlockAST &ast) {
    // 一个新的函数在 InsertFunc 时已经创建了形参/函数体作用域。
    if (start_of_new_func)
        start_of_new_func = false;
    else
        make_new_table();
    for (auto &item : ast.blockItemList) {
        item->is_inloop = ast.is_inloop;
        item->accept(*this);
    }
    delete_table();
}
```

左值访问部分也新增了较多检查。原有代码只检查了未定义变量，数组下标不是整数和非数组按数组访问还没有真正完成。修改后先查符号表，再检查每个下标表达式是否为整数；如果对非数组变量使用下标，或者数组下标维数超过声

明维数，就报 VisitVariableError。

```
Entry *entry = Lookup(*ast.id);
if (entry == nullptr || entry->is_func)
    ReportAndExit(ErrorType::VarUnknown, *ast.id);
for (auto &exp : ast.arrays) {
    if (exp) {
        exp->accept(*this);
        if (!Expr_int)
            ReportAndExit(ErrorType::ArrayIndexNotInt, *ast.id);
    }
}
if (!entry->is_array && !ast.arrays.empty()) {
    ReportAndExit(ErrorType::VisitVariableError, *ast.id);
}
if (entry->is_array && ast.arrays.size() >
    static_cast<size_t>(entry->array_length)) {
    ReportAndExit(ErrorType::VisitVariableError, *ast.id);
}
```

同时，修改后会根据已经使用的下标维度更新 current_type。如果数组没有被完全取到元素，当前表达式仍然是数组；如果维度用完，当前表达式才是普通变量。这部分是为了后面的函数参数匹配服务。

函数调用部分是修改最多的地方。修改后先特殊处理 SysY 运行时库函数，但仍然访问实参，以便发现实参内部的未定义变量等错误。对于普通函数，则检查函数是否存在、是否真的是函数、参数数量是否一致，以及每个实参与形参类型是否匹配。

```
Entry *entry = Lookup(*ast.id);
if (entry == nullptr || !entry->is_func)
    ReportAndExit(ErrorType::FuncUnknown, name);
if (entry->func_params.size() != ast.funcCParamList.size())
    ReportAndExit(ErrorType::FuncParamsNotMatch, name);
auto formal_it = entry->func_params.begin();
for (auto &actual : ast.funcCParamList) {
    actual->accept(*this);
    if (!SameParamType(*formal_it, current_type))
        ReportAndExit(ErrorType::FuncParamsNotMatch, name);
    ++formal_it;
}
```

最后,表达式类型推导也做了补充。原有代码中 `AddExpAST` 和 `MulExpAST` 只是递归访问子节点,没有根据左右操作数更新表达式类型。修改后保存左右表达式类型,只要有一边是 `float`,结果就是 `float`,否则为 `int`。这保证了数组下标检查、返回值检查和参数匹配能拿到正确的表达式类型。

```
if (has_left && has_right) {
    Entry right_type = current_type;
    current_type = Entry{};
    current_type.is_array = false;
    current_type.is_func = false;
    current_type.type = (left_type.type == TYPE::TYPE_FLOAT ||
                        right_type.type == TYPE::TYPE_FLOAT)
        ? TYPE::TYPE_FLOAT
        : TYPE::TYPE_INT;
    Expr_int = (current_type.type == TYPE::TYPE_INT);
}
```

5 中间代码生成（C++实现）

5.1 实验任务

在给定中间代码生成框架上，把 SysY AST 翻译为 LLVM IR 风格的中间表示。实现内容包括模块、类型、常量、全局变量、函数、基本块、指令打印以及 AST 到 IR 的遍历生成。

5.2 LLVM IR 生成器的实现

IR 框架中 Type、Value、Constant、Module、Function、BasicBlock、Instruction 等类描述 LLVM IR 的基本对象。Type::print 输出 i32、float、数组、指针和函数类型；Value 维护 use-list，Instruction 维护操作数；Function::set_instr_name 为匿名指令和基本块编号，保证打印出的 IR 可读且符合格式要求。

GenIR 负责从 AST 生成 IR。函数定义阶段先建立 FunctionType 和 Function，再创建 label_entry 基本块，把形参 alloca 后 store 到局部空间中，并建立统一返回基本块 label_ret。非 void 函数额外分配 retAlloca，return 语句把返回值存到该地址后跳转到统一返回块。

```
auto bb = new BasicBlock(module.get(), "label_entry", func);
builder->BB_ = bb;
for (int i = 0; i < (int)(paramNames.size()); i++) {
    auto alloc = builder->create_alloca(params[i]); // 分配形参空间
    builder->create_store(args[i], alloc);          // store 形参
    scope.push(paramNames[i], alloc);              // 加入作用域
}
// 创建统一 return 分支
retBB = new BasicBlock(module.get(), "label_ret", func);
```

语句生成中，赋值语句先以 requireLVal=true 访问左值，得到变量地址；再访问右值表达式，并根据左右类型插入 sitofp 或 fptosi 转换；最后用 create_store 写回。if/else 与 while 通过创建 BasicBlock 并发出条件跳转实现，break 跳到 whileFalseBB，continue 跳回 whileCondBB。


```
case ASS: {
    // 表明当前是赋值语句的左值处理
    requireLVal = true;
    ast.lVal->accept(*this); // 访问左值, 更新 recentVal 为左值的地址
    auto var = recentVal;    // 获取左值对应的变量 (地址)
    is_single_exp = true;
    ast.exp->accept(*this);  // 访问右值, 更新 recentVal 为右值的值
    auto expval = recentVal; // 获取右值的值
    // 类型转换, 确保左右值类型一致
    if (var->type->tid_ == Type::FloatTyID && expval->type->tid_ ==
        Type::IntegerTyID)
        // 如果左值是 float 类型, 右值是 int 类型, 进行 int 转 float
        expval = builder->create_sitofp(expval, FLOAT_T);
    else if (var->type->tid_ == Type::IntegerTyID && expval->type->tid_
        == Type::FloatTyID)
        // 如果左值是 int 类型, 右值是 float 类型, 进行 float 转 int
        expval = builder->create_fptosi(expval, INT32_T);
    // 使用 create_store 将右值存储到左值变量中
    builder->create_store(expval, var);
    break;
}
```

数组初始化是本实验中较复杂的部分。全局数组初始化使用 `globalInit` 递归处理嵌套大括号, 并通过 `mergeElements`、`finalMerge` 对齐维度和补零; 局部数组初始化通过 `localInit` 结合 `getelementptr` 逐元素 `store`。表达式生成区分常量求值和运行时值, 常量路径直接折叠为 `ConstantInt` 或 `ConstantFloat`, 运行时路径根据类型生成 `add/sub/mul/div/rem`、`icmp/fcmp`、`zext`、`sitofp` 等指令。

6 基于 LLVM 工具链的目标代码生成（RISCV64）

6.1 实验任务

基于 LLVM 工具链，把 LLVM IR 中间代码翻译成指定平台的目标代码。
本次完成 6-2 RISC-V64 目标代码生成，输出汇编或目标文件。

6.2 RISC-V64 目标代码生成器的实现

codegen.cc 首先用 `parseIRFile` 读取 IR 文件并构造 LLVM Module。随后初始化 LLVM 目标信息、目标、MC 层、汇编解析器和汇编打印器。目标三元组设置为 `riscv64-unknown-elf`，CPU 设置为 `generic-rv64`，再通过 `TargetRegistry::lookupTarget` 查找后端并创建 `TargetMachine`。

```
// 补充代码1 - 初始化目标
InitializeAllTargetInfos();
InitializeAllTargets();
InitializeAllTargetMCs();
InitializeAllAsmParsers();
InitializeAllAsmPrinters();

// 补充代码2 - 指定目标平台
auto target_triple = "riscv64-unknown-elf";
module->setTargetTriple(target_triple);
std::string error_string;
auto target = TargetRegistry::lookupTarget(target_triple, error_string);
if (!target) {
    errs() << error_string;
    return false;
}
auto cpu = "generic-rv64";
auto features = "";
TargetOptions opt;
auto RM = Optional<Reloc::Model>();
auto TheTargetMachine =
    target->createTargetMachine(target_triple, cpu, features, opt, RM);
module->setDataLayout(TheTargetMachine->createDataLayout());
```

生成文件名由 `getGenFilename` 根据输入 IR 文件名和 `CodeGenFileType`

华中科技大学实验报告

决定：汇编输出使用 .s，目标文件输出使用 .o。随后构造 raw_fd_ostream，创建 legacy::PassManager，并调用 addPassesToEmitFile 把目标机器的代码生成 pass 加入 pass manager，最后 pass.run(*module) 完成实际生成。

```
// 补充代码3 - 初始化 addPassesToEmitFile() 的参数
auto filename = getGenFilename(ir_filename, gen_filetype);
std::error_code EC;
raw_fd_ostream dest(filename, EC, sys::fs::OF_None);
if (EC) {
    errs() << "Could not open file: " << EC.message() << "\n";
    return false;
}
legacy::PassManager pass;
auto file_type = gen_filetype;
if (TheTargetMachine->addPassesToEmitFile(pass, dest, nullptr, file_type)) {
    errs() << "TheTargetMachine can't emit a file of this type";
    return false;
}
pass.run(*module);
dest.flush();
```

7 总结

本次实验按编译器前端到后端的顺序逐步完成：首先熟悉 gcc、clang、交叉编译和 qemu 等基础工具；随后用 ANTLR 完成词法和语法分析，建立从源代码到语法树、AST 的转换；再在 AST 上实现静态语义检查；之后生成 LLVM IR 风格中间代码；最后调用 LLVM 后端生成 RISC-V64 目标代码。

我对编译流程的分层有了更清晰的认识。词法和语法分析解决的是源程序结构化问题，语义检查在语法正确的基础上保证名字、类型和控制流使用符合语言规则，中间代码生成把高级语言语义落到可优化的 IR，目标代码生成则需要关注具体平台的 triple、CPU、数据布局 and 输出文件类型。

整个实验中，我认为收获最大的部分是作用域符号表、数组维度处理、类型转换和基本块控制流生成。这些内容分布在语义分析和 IR 生成的不同阶段，但本质上都要求编译器在遍历程序结构时准确保存上下文，并在合适的位置使用这些上下文信息。比如数组在语义检查阶段需要判断维度和下标是否合法，在 IR 生成阶段又需要根据维度计算地址；类型转换在语义检查阶段用于判断类型是否匹配，在 IR 生成阶段则需要生成 sitofp、fptosi 等具体指令。前后阶段虽然目标不同，但逻辑上是连续的。